

Joseph Miller

Traffic sign

detector

Chapter 1: Grand Overview

I would like to propose a traffic sign detector which would be a very basic full-stack project that implements a machine learning model to inference what traffic sign might be present in the photo. There are many reasons I would like to implement this. Today's world is full of computers doing really cool things. This is one of them. The eventual goal would be to fully implement a miniaturized machine learning model on an edge device, such as a jetson nano, and fully install it into an antique vehicle with a full glass panel dash display. Implementing this full stack website would be the first step in achieving this goal. While a lot of this stuff is addressed in this paper at a high level, many of these components will be subject to change if there are other requirements to implement.

The first and, likely simplest part of the whole project, is the user interface. I plan to implement a very basic front-end interface written in html, css, and Javascript. I will get to the specifics later but this gives me an opportunity to write up something quick and efficient while returning some metrics to the user (as well as picture inference). As a user, one is expected to do one of two things: either upload a photo or choose from a database of photos. Upon selection of the photo and the processing button clicked, the data flow diagram goes as follows under 2 scenarios:

Upload photo → display uploaded photo and add to data base → potential preprocessing/resizing if needed → inference engine processing → metrics processing → display information and inference to user

Display selected photo from database → add to data base → potential preprocessing/resizing if needed → inference engine processing → metrics processing → display information and inference to user

The second portion is the back-end processing/storage which are the meat and potatoes of this project. The general idea is to connect the front-end to a back-end that essentially connects all of the relevant components together. The back end will contain database connectivity along with a connection to the inference engine and will interface with the front end UI. Since we now have a working idea of how everything connects with each other, Kubernetes will be implemented as a way to orchestrate and meet demand of potential situational anomalies. The obvious anomaly is if a pod gets deleted or crashes, another will spawn in its place to make sure that everything keeps running smoothly. The whole brains of this project will be highly reliant on the back end and special care will be taken to ensure data gets processed in a correct fashion.

Chapter 2: Implementation

There is a data set on Kaggle that contains many pictures of traffic signs based on the United States, Great Britain, and Germany. Special care will be taken to ensure that only United States signs will be selected for training. Upon filtering the pictures, a notebook containing libraries from Keras and TensorFlow will be used to train the model. Data augmentation will also be used but this needs to be done carefully. Some forms of data augmentation include photo rotation and photo zoom. We want to probably avoid photo zoom as some signs might exist at the corner of some photos. If zoom data augmentation is used, it might cut off the actual traffic sign which means training on them would be like punishing the model for doing nothing wrong. The photo might contain a stop sign but the sign disappears if the zoom is too extreme. This will obviously affect inference accuracy. For the time being, however, a pretrained model will be used in its place to build a working back end model for this project.

The database of choice for this project will be MongoDB as it is simple to work with and easy to implement. Having it be a sql-less database, we can send query requests from the front end and get a very readable (and easily processable) JSON object. This JSON object will primarily consist of photo metadata since the database will not be responsible for storing the actual photos. There will be a separate directory (i.e. App/photos/photo1.jpg) that will handle the storage of photos that the user can choose from the front end to pass to the inference engine.

As described above in the first paragraph of this chapter, TensorFlow with Karas will be utilized for photo preprocessing as well as the inference engine services. Other services used will be the front end, back end, and database. NodePort will be used to expose the front end and clusterIP will be used for internal pod communication. Kubernetes will be utilized to orchestrate everything together. Rke2 will be used to seamlessly create nodes in which the pods will reside. I have chosen a design of 3 individual pods that will reside inside of an individual node. Desired state will be 3 replicas of each pod which will be outlined in the diagram on the next page. The first pod will contain the front end and back-end API, the second will contain the inference pod, and the third will contain the database pod. I have chosen this setup because it gives a basic and powerful understanding of how everything is stitched together. An additional metric to be considered is how this project is scaled. The inference engine will likely take a lot of computing power so I expect this to be the only scaled metric within this project if there is extra demand. I will pick an arbitrary result of 9 for this metric. The directory layout will follow something very similar to the following diagram on the next page:

App

Front end and API

Database

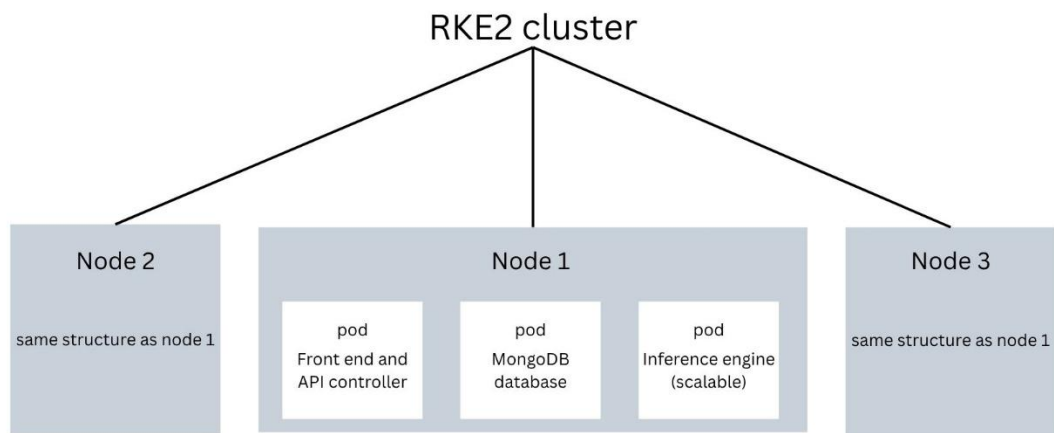
Inference Engine

Photos

The directory layout is intended to be simple and easily navigable. I have no intention of putting large file paths in this project.

Testing this project will be relatively simple as I would like to see a pod get restarted after intentionally deleting it. What I am concerned about is how the inference model would scale with many users. The obvious answer would be to scale the inference engine as high as it can go but it is preferable not to be wasteful in computer resources. Image size will be important since we do not want something too large or too small. Metrics for what defines these limitations are still being analyzed but a 12x12 png photo will obviously not make the cut. A 4k photo will definitely need some resizing and rescaling since the dimensions need to match what the inference engine is used to accepting. The project is considered done when all of these services are implemented correctly with no crashes and pods are returned to a desired state after deletion. The user interface is meant to be very simple so weird inputs to the data base will not be expected. File extensions for photos will be considered to make sure the app is accepting mainly .JPG and possibly .PNG.

Traffic sign image identifier



Resume on next page...

Joseph Miller
609-816-0183 joseph_miller@myyahoo.com

Innovative and collaborative IT professional with 6 years of expertise seeking an entry-level full-time IT or Software Development role.

EDUCATION

West Chester University of Pennsylvania, West Chester, PA
Bachelor of Science in Computer Science and Programming

WORK EXPERIENCE

Handi Crafters Opportunity Center, Thorndale, PA
Present

October 2023 –

Workshop Supervisor

- Manage fifteen individuals with disabilities in performing assembly tasks, ensuring consistent workflow and adherence to job specifications.
- Utilize positive coaching techniques to guide program participants toward achieving personal and vocational goals.
- Oversee the team to meet daily production deadlines, ensuring both individual and collective success.
- Accurately track and record daily production data, inputting information into relevant software applications.
- Lead initiatives in team meetings while managing planning and operational discussions.

St. Herman Of Alaska Orthodox Church, Glen Mills, PA
– Present

November 2021

Live Stream Specialist

- Provide overlays using HTML, CSS, and JavaScript to dynamically display information.
- Manage a full range of production techniques including sound recording, videography, lighting, and equipment to create high quality video content.
- Establish technology needs for livestreaming, including managing equipment and determining upgrades.

Advance Auto Parts, Frazer, PA
August 2022

July 2020 –

Assistant General Manager

- Excelled marketing and sales expectations by exceeding daily profit of \$4,000.
- Established strong knowledge of automotive abilities and systems to excel in customer satisfaction.
- Maintained parts catalogs and inventory to ensure customers timely access to merchandise.

Independent Contractor, Coatesville, PA
December 2018

July 2018 –

Private Python Tutor

- Engaged with students to complete homework assignments, identify skills, and improve overall performance.
- Reviewed core programming concepts including algorithms, conditional statements, and object-oriented programming.

KenCrest Services, Plymouth Meeting, PA
November 2017

August 2014 –

Personal Support Staff

- Assisted clients with daily living needs to maintain self-esteem and general wellness.
- Provided compassionate and patient-focused care to cultivate well-being.
- Assisted with money management and social integration to ensure a high quality of life.

SOFTWARE SKILLS

- Expert in Front End Web Development Technologies - HTML, CSS, JavaScript, and API calls
- Expert in Back End Technologies - Java, Python, C Sharp and SQL Databases
- Expert in Object Oriented Programming, Data Structures and Algorithms, Networking, and Input Validation
- Expert in version control technologies such as Git